EECS281 Mid

Step: * var. assignment
* arithmetic operation
* comparison * array index
* pointer ref * func. call <return

Complexity (B. Avg. Worst)

Big-O Def $\forall c>0, \exists n_0 \geq 0$
$f(n) \leq c g(n)$ for all $n \geq n_0$
$\Rightarrow f(n) = O(g(n))$

充分条件 $\lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty$

$f(n) = \Omega(g(n))$
$f(n) = O(g(n)), g(n) = O(f(n))$
$\Rightarrow f(n) = \Theta(g(n))$

Pseudocode

i=j; i==j; for i=0 to n;

Algorithm method(para)
Input: Output:

Recursion * tail recursion

Master Theorem

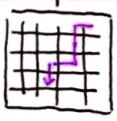
$$T(n) = \begin{cases} aT(n/b) + f(n) \\ 1, n=1 \end{cases}$$

最高次为c

$$T(n) = \begin{cases} \Theta(n^{\log_b a}), a>b^c \\ \Theta(n^c \log_2 n), a=b^c \\ \Theta(n^c), a<b^c \end{cases}$$

* cannot use if T(n) not monotonic; $f(n) \in \Theta(n^c)$

Stepwise linear search



Arrays in C/C++

* random access O(1)

String in C/C++.

strcat(s1, s2) (append)

strcpy(s1, s2)

char * strdup(const char *s)
↑记得free(p);

string s = s.c_str();

dynamic array

malloc() free()

new[] delete[]

e.g. int *array = new int[a]
delete[] array; array = nullptr;

Containers

Array class: the Big3+1

Constructor, Destructor

Copy constructor, "=".

* operator[] 要const和非const两个版本

* 2D+case:

Type &operator()(int x, int y){
}

* f(i++) 先运行 f(i);
f(++i) 先运行 ++i;

Linked List

	array	linked list
saccess	O(1)	O(n)
raccess	O(1)	O(1)
insert	O(n)	O(n)
append	O(n)	O(n)
bookkeeping	currsize maxsize	head (size) (tail)
memory	fixed	allocated

Complexity of delete

* doubly-linked: O(1)
* singly-linked: O(n)
法一: deletednode *prev)
use dummy for the first
法二: overwrite with next.
delete next * not for last

reverse 法一: insert
法二: change pointers dir.

Iterator:

- Input: read
- Output: write
- Forward: r&w
- Bidirectional: (default) ++; --; ↰ same
- Random: arithmetic + comparison
- Reverse:

Functor

struct SortByName{
bool operator()(...,...)const{
return xxx<xxx;
}
} sbn;
sort(x.begin(), x.end(), sbn);

utility swap, max, iota

Stack & Queue

stack: push, pop, top, size, empty.
all O(1)

queue: push, pop, front
O(1) if with shift
size, empty. all O(1)

deque: push_front.
pop_front, back.

front. back. size. empty

binary search

* to speed up. check <
before ==.

* lower/upper/equal_bound

set (union/intersection)

bitvector/bitset

vector<bool>

Sorting

STL sort: { insertion (small)
heap (big).

void **bubble sort**(item a[], int left, int right){
for(int i=left; i<right-1; i++){
bool swapped=0;
for(int j=right-1; j>i; j--){
if(a[j]<a[j-1]){
swapped=1;
swap(a[j-1], a[j]);
}
}
if(!swapped) break;
}
}

* quick if presorted.

void **selection sort**(...)
for(int i=left; i<right-1; i++){
int min=i;
for(int j=i+1; j<right; j++)
if(a[j]<a[min]) min=j
if(min!=i) swap(a[i], a[min]);
}
}

* great for small array
* not dependent on preorder

void **insertion sort**(){
for(int i=right-1; i>left; i--)
CompSwap(a[i-1], a[i]);
for(int i=left+2; i<right; i++){
Item v=a[i]; int j=i;
while(v<a[j-1]){
a[j]=a[j-1];
j--;
}
a[j]=v;
}
}

bin (bucket) sort

Counting sort

Array-based Heap

void fixUp(Item heap[], int k){
while(k>1 && heap[k/2] < heap[k]){
swap(heap[k], heap[k/2])
k/=2;
}
}
void fixDown(..., int size, ...){
while(2*k<=size){
int j=2*k;
if(j<size && heap[j]< heap[j+1]) j++;
if(heap[k]>=heap[j]) break;
swap(heap[k], heap[j]);
k=j;
}
}

Quicksort

partition(array);
quicksort(lh);
quicksort(rh);

Mergesort

mergesort(lh);
mergesort(rh);
merge(lh, rh);

	time	mem	stable?
b	n/n^2	1	✓
s	n^2	1	×
i	n/n^2	1	✓
m	nlogn	n	×
q	nlogn	logn.	×
h	nlogn	1	×.